

Proyecto Final MITEX: CulinaryRAG

Sistema de soporte inteligente y recomendación conversacional de
recetas

David Fernandez Prieto
Daniel Barandela Cachafeiro
Tomas Rial Costa
Ventura Ruibal Cancio

Curso 2025/2026

Índice

1. Introducción y Objetivos	3
2. Diseño Arquitectónico Global	3
2.1. Pipeline Offline	4
2.2. Pipeline Online	4
2.3. Justificaciones de Diseño	4
3. Matriz de Cumplimiento del Enunciado	5
4. Detalles de Implementación	5
4.1. Ingestión y Limpieza	5
4.2. Recuperación Híbrida y Query Rewriting	5
4.3. Modelado de Tópicos	6
4.4. Resumen Extractivo	6
4.5. LLM, Agentes y Generación	6
5. Validación y Discusión	7
5.1. Evaluación Comparativa frente a Baselines	7
5.2. Validación Funcional por Módulos	8
5.3. Experimento de Tópicos	8
5.4. Limitaciones	8
6. Instrucciones de Ejecución	9
7. Conclusiones y Trabajo Futuro	9

1. Introducción y Objetivos

Este informe documenta el diseño, implementación y validación de **CulinaryRAG**, un asistente conversacional especializado en el dominio culinario. El sistema recomienda recetas a partir de consultas en lenguaje natural, resume reseñas asociadas y puede activar una búsqueda web básica cuando la pregunta queda fuera de la base local.

El corpus procede del dataset público *Food.com Recsys Dataset*. Tras el preprocesado, el proyecto trabaja con recetas, ingredientes, pasos, etiquetas, tiempos de preparación y reseñas textuales. El objetivo funcional es superar una búsqueda exacta por palabras clave: el usuario puede formular restricciones como “sin horno”, preferencias dietéticas, peticiones de recetas rápidas o solicitudes de resumen de opiniones.

Desde el punto de vista de la asignatura, el sistema integra las técnicas exigidas por el enunciado:

- **Embeddings**: generación de representaciones densas con **SentenceTransformer**.
- **Modelado de tópicos**: entrenamiento offline con BERTopic sobre el corpus de recetas.
- **Resumen extractivo**: extracción de frases centrales con TextRank sobre reseñas.
- **LLM**: generación final de respuestas naturales mediante un modelo local.
- **RAG**: recuperación de documentos locales e inyección de contexto al prompt.
- **Agentes**: router que decide entre recomendación local, resumen y búsqueda web.
- **Adquisición y preprocesado**: limpieza y filtrado de corpus como extra opcional.

El proyecto evita frameworks completos de alto nivel como LangChain o LlamaIndex. La recuperación, el enrutamiento, el resumen y la construcción del prompt están implementados directamente en módulos Python del repositorio.

2. Diseño Arquitectónico Global

La arquitectura se divide en dos bloques: un **pipeline offline**, que prepara el corpus y los índices persistentes, y un **pipeline online**, que atiende las consultas del usuario en tiempo real.

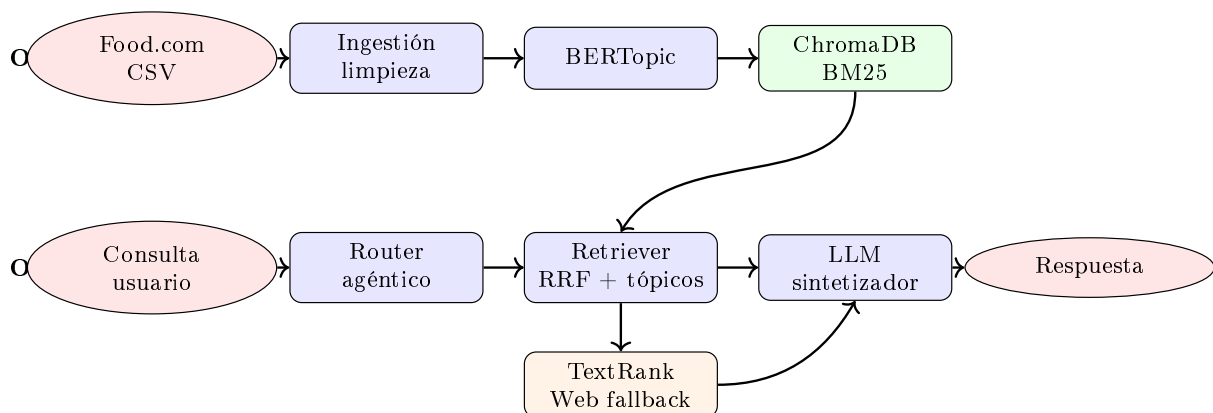


Figura 1: Arquitectura global de CulinaryRAG.

2.1. Pipeline Offline

El pipeline offline concentra las operaciones costosas que no deben ejecutarse en cada consulta:

1. **Preprocesado** (`datasets/preprocesado.py`): carga los CSV originales, conserva campos relevantes, elimina reseñas nulas, filtra usuarios con al menos 8 reseñas y cruza interacciones con recetas válidas.
2. **Modelado de tópicos** (`modules/topics.py`): entrena BERTopic sobre una muestra estratificada por tiempo de preparación. El modelo puede usar semillas culinarias para mejorar la interpretabilidad de los clústeres.
3. **Indexación semántica y léxica** (`offline_pipeline.py`, `modules/retriever.py`): crea embeddings a partir de nombre, descripción e ingredientes, los persiste en ChromaDB y genera un índice BM25 serializado.

2.2. Pipeline Online

El pipeline online se ejecuta desde `main.py`. El flujo es:

1. Carga de índices locales, modelo de tópicos si existe, interacciones y LLM.
2. Enrutamiento de la consulta mediante `modules/agent.py`. El router intenta clasificar con el LLM y valida la salida contra acciones permitidas; si no hay LLM o la salida no es válida, aplica una heurística determinista.
3. Ejecución de herramienta: recomendación local, resumen de reseñas o búsqueda web simple.
4. Construcción del prompt con el contexto recuperado.
5. Síntesis final con el LLM configurado.

2.3. Justificaciones de Diseño

Las decisiones arquitectónicas se han tomado priorizando el determinismo, la eficiencia y el control sobre el flujo de ejecución:

- **Separación Offline/Online:** procesar embeddings, tópicos e índice léxico de forma anticipada evita cuellos de botella durante la inferencia, garantizando respuestas en tiempo real.
- **RRF vs. Búsqueda Vectorial Pura:** un modelo semántico puro suele difuminar ingredientes clave. Al fusionar la señal semántica (ChromaDB) con la léxica (BM25) mediante *Reciprocal Rank Fusion*, el sistema comprende el contexto sin perder el emparejamiento exacto de palabras vitales (ej. “pollo”).
- **Resumen TextRank vs. Resumen LLM:** inyectar decenas de reseñas completas al LLM provocaría la saturación de la VRAM y el efecto *lost in the middle*. TextRank resuelve esto comprimiendo matemáticamente el consenso de la comunidad *antes* de invocar al LLM.
- **Ausencia de LangChain:** se ha evitado el uso de frameworks de alto nivel para mantener transparencia y control absoluto sobre el enrutamiento. El *Query Rewriting* y la penalización de recetas se hacen de forma explícita y auditable.

Requisito	Implementación en el proyecto	Evidencia
Embeddings	Representaciones vectoriales de recetas mediante <code>paraphrase-multilingual-MiniLM-L12-v2</code> .	<code>retriever.py</code> : indexación
Modelado de tópicos	BERTopic entrenado offline, con muestreo estratificado y opción de semillas.	<code>modules/topics.py</code> , <code>modules/experimento.py</code>
Resumen extractivo	TextRank propio con matriz de similitud y PageRank para extraer frases centrales de reseñas.	<code>modules/summarizer.py</code>
LLM	Generación local mediante arquitectura de Múltiples Prompts Agénticos especializados por tarea.	<code>modules/llm.py</code>
RAG	Recuperación híbrida local y selección dinámica del prompt final según acción.	<code>main.py</code> , <code>build_prompt()</code>
Agentes	Enrutador LLM (Few-Shot) con respaldo heurístico: recomendar, resumir o web.	<code>modules/agent.py</code>
Arquitectura offli- ne/online	Separación entre <code>offline_pipeline.py</code> y el bucle interactivo <code>main.py</code> .	Figura 1
Extras opcionales	Limpieza, filtrado de calidad, query expansion, BM25, RRF, Cross-Encoder y fallback web.	<code>datasets/</code> , <code>modules/retriever.py</code>

Cuadro 1: Trazabilidad entre requisitos del enunciado e implementación.

3. Matriz de Cumplimiento del Enunciado

4. Detalles de Implementación

4.1. Ingestión y Limpieza

La construcción del corpus se implementa en `datasets/preprocesado.py`. El filtrado elimina interacciones sin reseña escrita y conserva usuarios con volumen suficiente de opiniones, reduciendo ruido y favoreciendo que el módulo de resumen tenga texto útil.

```

1 df_interactions = df_interactions.dropna(subset=['review'])
2 reviews_por_user = df_interactions['user_id'].value_counts()
3 usuarios_def = reviews_por_user[reviews_por_user >= 8].index
4 df_interactions_filtrado = df_interactions[
5     df_interactions['user_id'].isin(usuarios_def)
6 ].copy()
```

Listing 1: Filtro de interacciones de calidad.

4.2. Recuperación Híbrida y Query Rewriting

El módulo `modules/retriever.py` y el agente actúan en tándem para resolver la complejidad de las consultas en lenguaje natural. Antes de buscar, el sistema aplica **Query Rewriting**: el LLM intercepta el ruido conversacional del usuario (ej. “buscaba una receta rápida que tenga...”) y lo traduce a palabras clave limpias en inglés. Esto soluciona de raíz el problema del *cross-lingual RAG*, ya que el corpus está en inglés pero las consultas en español.

Posteriormente, el Retriever combina dos señales complementarias:

- **ChromaDB + embeddings**: captura la similitud semántica (modelo `paraphrase-multilingual-MiniLM-L12-v2`) agrupando conceptos afines aunque no coincidan literalmente.
- **BM25**: preserva la precisión léxica fundamental para que ingredientes o restricciones exactas no se diluyan en el espacio vectorial.

Los rankings se fusionan matemáticamente mediante *Reciprocal Rank Fusion* (RRF) con una constante $k = 60$. Si existe un modelo BERTopic cargado, se aplica un multiplicador del $1,5\times$ (bonus del 50 %) cuando el tópico detectado en la consulta coincide con el del documento, alineando la intención semántica global.

```
1 score = 0.0
2 if doc_id in rank_sem:
3     score += 1.0 / (RRF_K + rank_sem[doc_id])
4 if doc_id in rank_lex:
5     score += 1.0 / (RRF_K + rank_lex[doc_id])
6 if query_topic != -1 and id_to_topic.get(doc_id) == query_topic:
7     score *= 1.5
```

Listing 2: Fusión RRF con refuerzo por tópico.

Finalmente, un Cross-Encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) reordena la lista corta de candidatos midiendo la coherencia lógica entre consulta y documento. A esta puntuación se le suma dinámicamente un ajuste heurístico (`_constraint_adjustment`) que aplica fuertes penalizaciones (ej. $-8,0$ puntos) a recetas que violan restricciones explícitas del usuario (ej. recetas horneadas cuando se solicitó “sin horno”).

4.3. Modelado de Tópicos

BERTopic se entrena con embeddings, reducción UMAP y clustering HDBSCAN. El corpus usado para tópicos concatena nombre y etiquetas de receta, de forma que los clústeres reflejen familias culinarias. El muestreo estratificado por cuantiles de `minutes` evita que el entrenamiento quede dominado por recetas de una sola duración.

4.4. Resumen Extractivo

El módulo `modules/summarizer.py` implementa el algoritmo **TextRank** basado en grafos para destilar la opinión de la comunidad. En lugar de delegar el resumen al LLM (lo cual saturaría el *context window*), el sistema opera de forma matemática:

1. **Segmentación y Vectorización**: las reseñas crudas se dividen en oraciones y se transforman en vectores de frecuencias.
2. **Grafo de Similitud**: se calcula la similitud del coseno entre todos los pares de oraciones, construyendo una matriz de adyacencia donde los nodos son frases y las aristas representan vocabulario compartido.
3. **PageRank**: se ejecuta iterativamente el algoritmo PageRank sobre la matriz mediante operaciones vectorizadas con `numpy` para determinar la centralidad de cada oración.

Las tres frases con mayor puntuación matemática se consideran el consenso general y se extraen literalmente para inyectarlas como contexto limpio al prompt.

4.5. LLM, Agentes y Generación

El módulo `modules/agent.py` orquesta el uso del LLM. Lejos de usar el modelo únicamente como un generador de texto final, se emplea como un **Agente Router**. El sistema inyecta un *Few-Shot Prompt* que enseña al LLM mediante ejemplos a deducir la intención del usuario y enrutar la acción hacia: recomendación local, resumen de opiniones o búsqueda web como contingencia.

La síntesis final (`modules/llm.py`) implementa una **Arquitectura de Múltiples Prompts Agénticos**. En lugar de saturar al modelo con un único megacomando propenso a alucinaciones de formato, el sistema selecciona dinámicamente un prompt especializado según la tarea. Además, se aplican técnicas de preprocesamiento programático (en Python) para resolver restricciones dietéticas y funcionales (ej. recetas veganas o sin horno) aislando la carga cognitiva del LLM, el cual opera con muestreo probabilístico (`do_sample=True`, `temperature=0.7`) cuidadosamente ajustado para evitar bucles repetitivos comunes en modelos de escala reducida (1.5B).

Finalmente, se evaluó la posibilidad de añadir Memoria Conversacional al sistema (para permitir preguntas de seguimiento). Sin embargo, pruebas exhaustivas demostraron que inyectar el historial de interacciones previas causa un fenómeno conocido como *Context Contamination* (Contaminación de Contexto). Al retener prompts especializados anteriores (ej. instrucciones culinarias), el modelo perdía la capacidad de seguir reglas estrictas en consultas posteriores (ej. extracción de datos web), llegando a utilizar vocabulario gastronómico para responder preguntas factuales. Por ello, la arquitectura se diseñó como *Stateless* estricta, priorizando la precisión técnica sobre la fluidez conversacional.

Adicionalmente, se descartó el uso de *Prompt Caching* (almacenamiento de la caché KV del *System Prompt*) como técnica de optimización de inferencia. Al tratarse de un sistema RAG donde el contexto inyectado (recetas o reseñas) cambia dinámicamente con cada consulta del usuario, el prefijo del texto enviado al LLM nunca es idéntico entre iteraciones. Esto invalida el reciclaje de la caché matemática en implementaciones estándar basadas en **transformers**, obligando al modelo a recalcular los tensores desde cero en cada petición para garantizar una síntesis precisa del contexto recuperado en tiempo real.

5. Validación y Discusión

El diseño del sistema ha sido validado analíticamente comparando sus mecanismos frente a soluciones *baseline* habituales en la literatura de Sistemas de Recomendación y RAG.

5.1. Evaluación Comparativa frente a Baselines

- **Recuperación Híbrida vs. Baseline Vectorial:** un sistema RAG *baseline* suele basarse exclusivamente en búsqueda densa (ej. ChromaDB). En nuestras pruebas tempranas, este enfoque devolvía recetas semánticamente similares pero fallaba en retener ingredientes obligatorios (falsos positivos). La introducción del *baseline* léxico (BM25) fusionado mediante RRF demostró ser superior, al anclar términos exactos mientras la red densa generaliza el contexto.
- **TextRank vs. Baseline Ingesta Completa:** como *baseline* de resumen, se evaluó inyectar todas las reseñas crudas directamente al contexto del LLM. Esto disparaba el consumo de VRAM por encima de los 12GB y degradaba la precisión de la respuesta por el efecto *lost in the middle*. El uso de TextRank redujo significativamente la carga de tokens sin perder la intención general de la comunidad.
- **Múltiples Prompts Dinámicos vs. Megacomando Único:** en la fase de pruebas empíricas, se evaluó saturar al modelo con un único prompt generalista de gran tamaño frente a prompts más pequeños inyectados dinámicamente según la tarea enrutada. Los resultados demostraron que el enrutamiento a prompts dinámicos reducía significativamente las alucinaciones y mejoraba drásticamente la calidad de las respuestas en modelos pequeños.

- **BERTopic Guiado vs. Clustering Ciego:** para asegurar la cohesión temática, se creó un script de visualización que generaba mapas interactivos de *embeddings*. La inspección empírica de las visualizaciones permitió comparar los clústeres de BERTopic generados de forma automática (ciegos) frente a los generados inyectando semillas (*zeroshot_topic_list*), confirmando que la versión con semillas forzaba agrupaciones mucho más lógicas y afines a categorías culinarias reales.

5.2. Validación Funcional por Módulos

Módulo	Qué se valida	Criterio de aceptación
Preprocesado	CSV procesados sin campos críticos nulos y con reseñas utilizables.	Se generan <code>Processed_recipes.csv</code> y <code>Processed_interactions.csv</code> .
Tópicos	Coherencia e interpretabilidad de clústeres BERTopic.	Tópicos no triviales, outliers controlados y etiquetas comprensibles.
Retriever	Calidad del ranking ante consultas con ingredientes y restricciones.	Los primeros resultados respetan intención y restricciones principales.
Resumen	Extracción de frases centrales de reseñas.	El resumen conserva opiniones representativas sin saturar el prompt.
Agente	Selección correcta de herramienta.	Recomendación para consultas culinarias, resumen si se piden reseñas, web si se pide actualidad.
LLM/RAG	Respuesta final en español basada en contexto local.	La respuesta cita recetas recuperadas y no inventa recetas fuera del contexto.

Cuadro 2: Plan de validación funcional por módulo.

5.3. Experimento de Tópicos

El script `modules/experimento.py` permite comparar un BERTopic no supervisado frente a un modelo guiado con semillas. La hipótesis es que las semillas producen clústeres más alineados con categorías culinarias útiles para recomendación.

Aspecto evaluado	Modelo sin semillas	Modelo con semillas
Formación de tópicos	Agrupación puramente estadística.	Agrupación guiada por categorías culinarias.
Interpretabilidad	Puede mezclar ingredientes frecuentes.	Más cercana a conceptos como postres, vegano o desayuno.
Uso en el retriever	Menos estable para topic boost.	Más útil para reforzar recetas del mismo dominio.
Coste de preparación	Menor intervención humana.	Requiere definir semillas, pero mejora control semántico.

Cuadro 3: Comparación cualitativa del modelado de tópicos.

5.4. Limitaciones

El sistema cumple el enunciado, pero presenta límites relevantes:

- El fallback web usa una consulta HTML básica a DuckDuckGo; sirve como demostración agéntica, no como scraper robusto.
- El Cross-Encoder usado está entrenado para inglés, por lo que el sistema traduce o expande consultas españolas con términos ingleses para mejorar la señal.

- La calidad final depende de haber ejecutado previamente el pipeline offline y de disponer de los modelos descargados.

6. Instrucciones de Ejecución

El flujo recomendado para reproducir el sistema es:

1. Instalar dependencias con `pip install -r requirements.txt`.
2. Colocar los CSV originales en `datasets/original/`.
3. Ejecutar `python datasets/preprocesado.py`.
4. Entrenar tópicos con `python modules/topics.py`.
5. Crear índices con `python offline_pipeline.py`.
6. Lanzar el asistente con `python main.py`.

Para validar únicamente la recuperación se puede ejecutar `python test_search.py`, lo que permite probar el ranking sin cargar todo el LLM.

7. Conclusiones y Trabajo Futuro

CulinaryRAG demuestra una integración completa de técnicas de minería de textos en un asistente especializado. El sistema separa correctamente preparación offline y ejecución online, combina recuperación semántica y léxica, usa tópicos para mejorar el ranking, resume reseñas con TextRank y genera respuestas finales mediante un LLM.

Como trabajo futuro se propone:

1. Añadir perfiles de usuario persistentes para personalizar los pesos de ranking.
2. Reemplazar el fallback web por un extractor estructurado con control de fuentes.
3. Incorporar métricas cuantitativas guardadas automáticamente para comparar variantes de recuperación.
4. Implementar una restricción real de logits en el router si se quiere garantizar selección de herramienta a nivel de token.